



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Different Approaches for Lift Coefficient Prediction on an Airfoil: Classical CFD and Deep Learning Techniques

PROJECT FOR THE COURSE OF ADVANCED METHODS FOR SCIENTIFIC COMPUTING

Giulio Enzo Donninelli, Alessia Rigoni, Vittorio Sironi and Alessandro Verrengia

Advisor:
Prof. Luca Dedè

Academic year:
2025-2026

Abstract: This work presents an integrated computational pipeline for rapid aerodynamic analysis, bridging high-fidelity simulations with a data-driven surrogate model. We extended the **LifeX** C++ finite element library with a Reynolds-Averaged Navier-Stokes (RANS) solver using the Spalart-Allmaras turbulence model. This solver generated a reference dataset on the CINECA Leonardo HPC cluster, which was used to train a Convolutional Neural Network (CNN) built from scratch in C++. The CNN predicts lift coefficients from a Signed Distance Function (SDF) geometry representation and is trained in parallel using MPI. The model achieves a validation Mean Squared Error of 0.0104 and a $35\times$ speedup over direct simulation, providing a robust framework for efficient aerodynamic design.

1. Introduction

Aerodynamic design balances a trade-off between the demand for physical fidelity and the need for rapid design iteration. High-fidelity Computational Fluid Dynamics (CFD) simulations, such as those based on the Reynolds-Averaged Navier-Stokes (RANS) equations, offer unparalleled accuracy in predicting complex flow phenomena. However, their significant computational cost makes them impractical for design-space exploration or real-time analysis. Conversely, data-driven surrogate models, especially deep learning architectures like Convolutional Neural Networks (CNNs), can provide near-instantaneous predictions but are entirely dependent on the quality and availability of the high-fidelity data used for their training.

This project develops a pipeline that integrates both methods. Inspired by the framework of Cuozzo et al. (2026), we first build a high-performance CFD solver to generate reference-quality aerodynamic data. We then use this data to train a custom CNN surrogate model. By establishing this workflow for clean symmetric airfoils, we made a base idea for future extensions to more complex iced geometries.

The document is structured to follow this approach, beginning with the development of the high-fidelity simulation framework. Section 2 (**Navier-Stokes Simulation**) details the extension of the **LifeX** C++ finite element library to handle external aerodynamic flows. We describe the integration of the one-equation Spalart-Allmaras RANS model, the adaptation of the solver to two-dimensional domains, the implementation of specialized boundary conditions and the development of utilities for computing aerodynamic forces. This section also provides a comprehensive overview of the High-Performance Computing (HPC) workflow established on the CINECA Leonardo cluster.

Building upon the dataset generated by these simulations, Section 3 (**Convolutional Neural Network**), presents the design, implementation and training of our surrogate model. A key contribution of this work is

the development of a CNN entirely from scratch in C++ , designed to predict the lift coefficient from an airfoil’s geometric representation along with the angle of attack and Reynolds number. We elaborate on the mathematical foundations of the network architecture, the physics-informed loss function that regularizes training and the implementation of distributed-memory parallelism using MPI, enabling efficient execution on HPC hardware.

2. Navier-Stokes Simulation

The main goal of this section of the project is to extend the **LifeX** library with new features, making it a more general-purpose and high-performance simulation framework. Specifically, the work focuses on integrating one-equation **RANS turbulence model** (Spalart-Allmaras) into the existing incompressible **Navier-Stokes solver** (NS), enabling the simulation of turbulent flows in external aerodynamic configurations.

2.1. LifeX Implementation

As detailed by Africa et al. (2024), **LifeX** is an open-source C++ library built on top of **deal.II**, originally designed for life-science applications such as cardiac and cardiovascular fluid dynamics. While the library provides a robust and well-structured finite element infrastructure, its original focus on biomedical applications necessitated several non-trivial adaptations to extend it to external aerodynamic flows.

This work addressed four primary challenges:

- adapting the solver and utility functions, originally designed for three-dimensional problems, to two-dimensional domains (Section 2.1.1);
- integrating the one-equation Spalart-Allmaras (SA) RANS model to account for turbulent effects (Section 2.1.2);
- implementing specialized conditions for aerodynamic simulations, including wall functions and far-field boundaries (Section 2.1.3)
- developing a residual-based post-processing utility to compute lift and drag forces (Section 2.1.4).

The following sections detail the implementation of each of these components.

2.1.1. 2D Implementation

While the **LifeX FluidDynamics** solver is templated on the spatial dimension **dim** through the **deal.II** convention, its original test cases and utilities were primarily designed for three-dimensional domains. Extending the framework to 2D involved adapting three key components: the finite element function space, the calculation of vorticity and the computation of the wall distance field.

Function space and triangulation

The SA solver uses Q_1 **finite elements** (**FE_Q** of degree 1) on the same triangulation as the NS solver:

$$\tilde{v}_h \in V_h = \{ v \in H^1(\Omega) \mid v|_{\partial\Omega_D} = g_D \} \quad (1)$$

On the implementation side, the SA Degree of Freedom (DoF) handler (**dof_handler_sa_**) is independent from the NS one (**dof_handler**), in a way that the two systems have disjoint degrees of freedom without altering the block structure of the Navier-Stokes system. The shared triangulation object is the key architectural choice that allows both DoF handlers to be iterated in lockstep during matrix assembly.

Vorticity in 2D

The modified vorticity $\tilde{S}$ entering the SA production term requires the vorticity magnitude $S = |\boldsymbol{\omega}|$. In three dimensions, this is computed as $S = |\nabla \times \mathbf{u}|$.

In a 2D context, it simplifies to the absolute value of the scalar curl:

$$S = \left| \frac{\partial u_y}{\partial x} - \frac{\partial u_x}{\partial y} \right| \quad (2)$$

which is extracted from the velocity gradient tensor $\nabla \mathbf{u}^{n+1}$ interpolated at quadrature points via `fe_values_ns_.get_function_gradients`.

Wall distance in 2D

The wall distance $d(\mathbf{x})$ is computed once during `setup()` via a parallel algorithm. Each MPI rank collects the barycenters of boundary faces tagged as `prm_wall_tag_` (airfoil surface), serializes them into a flat `double` array of size $N_{\text{face,loc}} \times 2$, and participates in a global `MPI_Allgatherv` to build the complete set $\{\mathbf{c}_w\}_{w=1}^{N_{\text{face,tot}}}$.

The distance at every locally owned SA node i is then:

$$d_i = \max\left(\min_w \|\mathbf{x}_i - \mathbf{c}_w\|_2, 10^{-12}\right) \quad (3)$$

where the lower clamp prevents division by zero for nodes sitting exactly on the wall boundary.

The result is stored in `wall_distance_ghosted_`, a Trilinos ghosted vector indexed on `dof_handler_sa_`, following the standard pattern:

1. write into an owned vector and call `compress(VectorOperation::insert)`.
2. copy owned $\rightarrow$ ghosted to propagate ghost entries across MPI ranks.

The overall complexity per rank is $O(N_{\text{face,tot}} \cdot N_{\text{nodes,loc}})$, which is acceptable for the 2D meshes used in the airfoil example.

2.1.2. Turbulence Model

To enable the simulation of turbulent flows, the framework was extended with the one-equation SA RANS model, introduced by Spalart and Allmaras (1992). This model introduces a transport equation for a scalar variable known as the *modified kinematic viscosity*, $\tilde{\nu}$:

$$\boxed{\frac{\partial \tilde{\nu}}{\partial t} + \mathbf{u} \cdot \nabla \tilde{\nu} = \underbrace{c_{b1} \tilde{S} \tilde{\nu}}_{\text{production}} - \underbrace{c_{w1} f_w \left(\frac{\tilde{\nu}}{d}\right)^2}_{\text{destruction}} + \underbrace{\frac{1}{\sigma} \left[\nabla \cdot ((\nu + \tilde{\nu}) \nabla \tilde{\nu}) + c_{b2} |\nabla \tilde{\nu}|^2 \right]}_{\text{diffusion + cross-diffusion}}} \quad (4)$$

The model constants (Table 1) follow the standard set by **NASA Turbulence Modeling Resource (TMR)** without the f_{t2} trip term.

Table 1: Spalart-Allmaras model constants based on the NASA TMR standard.

Constant	Value	Role
c_{b1}	0.1355	production coefficient
c_{b2}	0.622	cross-diffusion coefficient
σ	2/3	turbulent Prandtl number
κ	0.41	von Kármán constant
c_{v1}	7.1	viscous damping
c_{w1}	≈ 3.2391	destruction (derived from log-law consistency)
c_{w2}	0.3	regularization in f_w
c_{w3}	2.0	regularization in f_w

Closure functions

The model relies on a set of non-linear closure functions to relate $\tilde{\nu}$ to the turbulent eddy viscosity. Defining the viscosity ratio $\chi = \tilde{\nu}/\nu$, these functions are:

$$f_{v1}(\chi) = \frac{\chi^3}{\chi^3 + c_{v1}^3}, \quad f_{v2}(\chi) = 1 - \frac{\chi}{1 + \chi f_{v1}(\chi)},$$

$$\tilde{S} = \max\left(S + \frac{\tilde{\nu} f_{v2}}{\kappa^2 d^2}, 10^{-10}\right),$$

$$r = \min\left(\frac{\tilde{\nu}}{\tilde{S} \kappa^2 d^2}, 10\right), \quad g = r + c_{w2}(r^6 - r), \quad f_w(g) = g \left(\frac{1 + c_{w3}^6}{g^6 + c_{w3}^6}\right)^{1/6},$$

The final turbulent eddy viscosity, ν_t , which modifies the momentum equations, is then computed as:

$$\nu_t = \tilde{\nu} f_{v1}(\chi).$$

Time discretization and operator splitting.

To solve the coupled Navier-Stokes and Spalart-Allmaras system, this is advanced at each time step using a **first-order Lie splitting** scheme (splitting error $O(\Delta t)$). This approach is consistent with the first-order BDF1 time integration used for both solvers.

Given the state $(\mathbf{u}^n, p^n, \tilde{\nu}^n)$ at time t^n , the solution at t^{n+1} is obtained in two steps:

1. the **NS step** solve the momentum and continuity equations with the effective viscosity frozen at $\tilde{\nu}^n$:

$$\rho \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + \rho(\mathbf{u}^* \cdot \nabla) \mathbf{u}^{n+1} - \nabla \cdot (2\mu_{\text{eff}}^n \nabla^S \mathbf{u}^{n+1}) + \nabla p^{n+1} = \mathbf{0},$$

where $\mu_{\text{eff}}^n = \mu + \nu_t(\tilde{\nu}^n)$ is *explicit* with respect to the SA variable;

2. the **SA step** solve the transport equation using the velocity field $\mathbf{u}^{n+1}$ just computed and a semi-implicit linearization:

$$\frac{\tilde{\nu}^{n+1} - \tilde{\nu}^n}{\Delta t} + \mathbf{u}^{n+1} \cdot \nabla \tilde{\nu}^{n+1} = c_{b1} \tilde{S}^n \tilde{\nu}^{n+1} - c_{w1} f_w^n \frac{\tilde{\nu}^n}{d^2} \tilde{\nu}^{n+1} + \frac{\nu + \tilde{\nu}^n}{\sigma} \Delta \tilde{\nu}^{n+1} + \frac{c_{b2}}{\sigma} |\nabla \tilde{\nu}^n|^2.$$

The linearization strategy is the following:

- the **production** is treated implicitly using a Picard iteration. $\tilde{S}^n$ is frozen and $\tilde{\nu}^{n+1}$ appears linearly in the matrix (stabilizing for $\tilde{S}^n > 0$);
- the **destruction** is linearized around $\tilde{\nu}^n$. The coefficient $c_{w1} f_w^n \tilde{\nu}^n / d^2$ enters as a positive reaction term, ensuring coercivity;
- about the **Diffusion**, the coefficient $(\nu + \tilde{\nu}^n) / \sigma$ is explicit while $\Delta \tilde{\nu}^{n+1}$ is implicit;
- about the **Cross-diffusion**, $c_{b2} / \sigma |\nabla \tilde{\nu}^n|^2$ is fully explicit and moved to the right-hand side.

Weak formulation and SUPG stabilization

To handle the advection-dominated nature of the SA equation, the weak form is stabilized using the **Streamline-Upwind Petrov-Galerkin** (SUPG) method.

Multiplying by the SUPG test function $\psi_h = v_h + \tau_{\text{SUPG}} \mathbf{u} \cdot \nabla v_h$ and integrating over Ω , the discrete linear system components become:

$$A_{ij}^K = \int_K \left[\psi_i \frac{\phi_j}{\Delta t} + \psi_i (\mathbf{u} \cdot \nabla \phi_j) + D \nabla \phi_i \cdot \nabla \phi_j - \psi_i c_{b1} \tilde{S}^n \phi_j + \psi_i c_{w1} f_w^n \frac{\tilde{\nu}^n}{d^2} \phi_j \right] dK,$$

$$b_i^K = \int_K \psi_i \left[\frac{\tilde{\nu}^n}{\Delta t} + \frac{c_{b2}}{\sigma} |\nabla \tilde{\nu}^n|^2 \right] dK,$$

where $D = (\nu + \tilde{\nu}^n) / \sigma$.

The stabilization parameter τ_{SUPG} is defined locally for each cell K as:

$$\tau_{\text{SUPG}} = \frac{1}{\sqrt{\left(\frac{2}{\Delta t}\right)^2 + \left(\frac{2|\mathbf{u}|}{h_K}\right)^2 + \left(\frac{4D}{h_K^2}\right)^2}},$$

where h_K is the minimum vertex distance of cell K .

Physical bound enforcement.

After each linear solve, a physical bound is enforced on the owned vector before updating the ghosted copy (the ordering `compress(VectorOperation::insert)` $\rightarrow$ ghosted copy is mandatory):

- negative values ($\tilde{\nu} < 0$) are clamped to 0.0, since turbulent viscosity cannot be negative (SA physical bound);
- NaN values ($\tilde{\nu} = \text{NaN}$) are reset to $\tilde{\nu}_\infty$ (freestream value) rather than zero, so that the affected cell remains active and self-corrects over subsequent time steps, avoiding a permanently "dead" region in the domain.

Adaptive time-stepping and bisection.

The bisection mechanism of `FluidDynamics` is fully supported.

`try_timestep_with_bisection` is declared `virtual` in the base class, and `FluidDynamicsSA` overrides it by calling `sa_solver.take_snapshot()` before each attempt and `sa_solver.restore_snapshot()` on failure, ensuring that the SA state $\tilde{\nu}^n$ is rolled back consistently at every bisection level.

2.1.3. Boundary Conditions

The implementation handles three distinct boundary condition types (the airfoil wall, the farfield inlet and the outlet boundaries) imposed on the SA variable each with a different physical motivation.

Wall-resolved (low-Re regime).

A homogeneous Dirichlet condition is applied on the airfoil surface: $\partial\Omega_{\text{wall}}$:

$$\tilde{\nu} = 0 \quad \text{on } \partial\Omega_{\text{wall}}. \quad (5)$$

This is the physically correct condition in the viscous sublayer, as the viscosity ν_t must vanish as the distance to the wall $d \rightarrow 0$.

Wall functions (high-Re regime).

When the first mesh node is at $y^+ > 5$ (unresolved sublayer), a non-homogeneous Dirichlet condition is derived from the **log-law**:

1. the friction velocity is estimated via Schlichting's flat-plate formula:

$$C_f = \frac{0.026}{Re^{0.143}} \quad \text{and} \quad u_\tau = U_\infty \sqrt{C_f/2}$$

2. the wall-normal distance of the first node is converted to wall units:

$$y^+ = \frac{d_{\text{node}} u_\tau}{\nu}.$$

3. a switch criterion at $y_{\text{sw}}^+ = 11.06$ separates the sublayer from the log-layer:

$$\tilde{\nu}_{\text{wall}} = \begin{cases} 0 & \text{if } y^+ < y_{\text{sw}}^+, \\ \text{root of } \nu_t(\tilde{\nu}) = \nu \kappa y^+ & \text{otherwise.} \end{cases}$$

4. the inversion of $\nu_t(\tilde{\nu}) = \tilde{\nu} f_{v1}(\tilde{\nu}/\nu)$ is performed with a **fixed-point iteration**:

$$\tilde{\nu}^{(k+1)} = \frac{\nu \kappa y^+}{f_{v1}(\tilde{\nu}^{(k)}/\nu)}, \quad k = 0, \dots, 4,$$

which converges in 5 iterations due to the monotonicity of f_{v1} .

Inlet and farfield.

A Dirichlet condition with freestream value is imposed:

$$\tilde{\nu} = \tilde{\nu}_\infty \in [3\nu, 5\nu] \quad \text{on } \partial\Omega_{\text{inlet}} \cup \partial\Omega_{\text{farfield}}. \quad (6)$$

Outlet.

No condition is explicitly imposed: the natural Neumann condition $\partial_n \tilde{v} = 0$ arises automatically from the boundary term produced by integration by parts of the diffusion operator in the weak form.

2.1.4. Aerodynamic Coefficients (Residual-Based Approach)

Lift and drag coefficients are computed at the end of each time step via a **residual-based boundary force method**, using the `FluidDynamics::compute_force(tag)` method available in `LifeX`, which integrates the full stress tensor over the boundary face with the prescribed tag (airfoil wall):

$$\mathbf{F} = \int_{\partial\Omega_{\text{wall}}} \boldsymbol{\sigma} \cdot \mathbf{n} d\Gamma, \quad \boldsymbol{\sigma} = -p \mathbf{I} + 2\mu_{\text{eff}} \nabla^S \mathbf{u}. \quad (7)$$

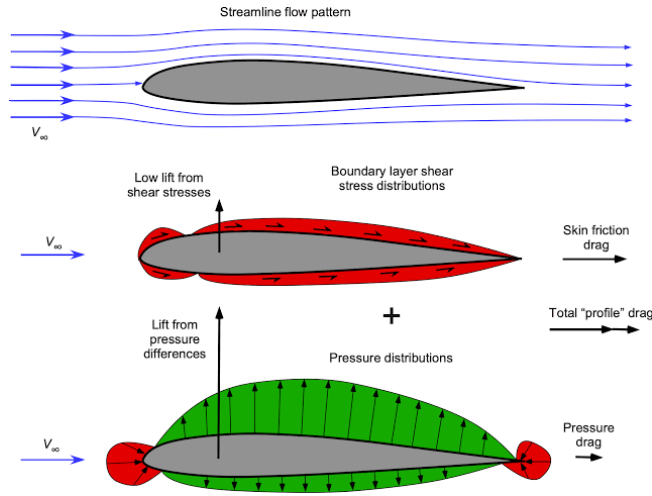


Figure 1: Aerodynamic forces on an airfoil. Source: Eagle Pubs

As illustrated in Figure 1, the Cartesian force components (F_x, F_y) are projected into the aerodynamic reference frame using the angle of attack α :

$$D = F_x \cos \alpha + F_y \sin \alpha, \quad L = -F_x \sin \alpha + F_y \cos \alpha. \quad (8)$$

The forces are normalized by the dynamic pressure q_∞ and the reference surface S_{ref} (which corresponds to the chord length in 2D simulations):

$$q_\infty = \frac{1}{2} \rho U_\infty^2, \quad C_L = \frac{L}{q_\infty S_{\text{ref}}}, \quad C_D = \frac{D}{q_\infty S_{\text{ref}}}. \quad (9)$$

Implementation details

This procedure is encapsulated in a header-only utility class `AerodynamicCoefficientsEvaluator` that exposes a single method:

```
1 compute_coefficients(tag, density, U_inf, S_ref, alpha)
```

Listing 1: Method from `AerodynamicCoefficientsEvaluator`.

It is registered as a callback via `FluidDynamics::set_end_timestep_callback`, so it executes automatically after each successful NS time step. Results are then appended to a CSV file (`force.csv`) with columns `time`, `F_lift`, `F_drag`, `C_L`, `C_D`.

2.1.5. Configuration

The turbulence model is selected via the parameter "Turbulence model": "Spalart-Allmaras" in the JSON configuration file.

For backward compatibility, FluidDynamicsSA also accepts the legacy value "None", overriding it internally with `Turbulence::SpalartAllmaras` after `parse_parameters`.

The enum class `Turbulence` in `fluid_dynamics.hpp` was extended accordingly:

```
1 enum class Turbulence { None, VMS_LES, SpalartAllmaras };
```

Listing 2: Extension of the class `Turbulence`.

2.2. Computational framework and execution workflow

This section details the computational environment and the end-to-end workflow established for the simulation.

2.2.1. HPC platform and software stack

All simulations were executed on **Leonardo** at CINECA (Bologna), targeting the `dcgp_usr_prod` partition (Intel Sapphire Rapids nodes, 112 cores per node) under the EuroHPC allocation EUHPC_D35_025, with job submission managed by the SLURM scheduler.

The simulation framework is **LifeX** 2.1, introduced in Section 2.1, compiled against `deal.II` 9.7.1 with **Trilinos** as the linear-algebra backend, MPICH 4.0 for distributed-memory parallelism, and HDF5 1.14.5 for field output in XDMF format. Meshes are generated externally with Gmsh and read by **LifeX** in MSH 2.2 format.

The full software stack is delivered to the compute nodes through an Apptainer image (`lifex_env.sif`) maintained by the **LifeX** developers, which removes the need for any host-level `module load` of HPC libraries; the build procedure and the rationale behind this choice are detailed in Section 2.2.2.

The resources allocated for each run of the simulation campaign are summarized in Table 2.

Component	Specification
Cluster	Leonardo @ CINECA (EuroHPC)
Partition	<code>dcgp_usr_prod</code>
CPU	Intel Sapphire Rapids
Cores per node	112
MPI ranks per simulation	8
Wall-time per run	up to 24 h
Software stack	LifeX 2.1 / <code>deal.II</code> 9.7.1 / Trilinos / MPICH 4.0 / HDF5 1.14.5
Container runtime	Apptainer 1.4.5 (image <code>lifex_env.sif</code>)

Table 2: Computational resources for the simulation campaign on Leonardo.

2.2.2. Containerization and build process

The compute nodes of Leonardo do not expose a pre-configured HPC software stack through the `module` command, which only loads base system libraries. A native build of **LifeX** on the cluster would therefore require manually compiling `deal.II` 9.7 together with all its required backends¹, an operation that is both fragile and not portable across nodes. The entire toolchain was therefore packaged into an Apptainer image and executed through the container runtime available on the compute nodes (Apptainer 1.4.5), so that the build environment is identical to the execution environment.

¹**Trilinos** (used by `LinearAlgebraTrilinos` and the `sacado_dfad` AD operators), PETSc, p4est, VTK, Boost, and HDF5

Identifying an image that simultaneously satisfies all LifeX dependencies required iterating over several candidates, summarized in Table 3, encountering a series of incompatibilities:

- the Spack stack pre-installed on Leonardo provides dealii@9.6.2, but only with the `~trilinos` variant, which is incompatible with LifeX 2.1;
- the official dealii/dealii Docker images expose deal.II 9.7 with all required backends, but lack the `boost_filesystemConfig.cmake` component required by the LifeX CMake configuration;
- an earlier MOX mk container shipping deal.II 9.5.1 fails to compile LifeX 2.1 due to API changes in `get_mpi_communicator`, `CellStatus`, and `AffineConstraints` that occurred upstream between 9.5 and 9.7

The image finally adopted, `mk-dealiiimaster_latest.sif`, is the MOX MK container with the `deal.II` module tracking the upstream master branch (deal.II 9.7), together with matching `vtk`, `boost`, `trilinos`, `petsc`, `p4est`, and `hdf5` modules, activated inside the container by sourcing `/u/sw/etc/profile` and loading the relevant modules.

Stack	Components	Outcome
Spack dealii@9.6.2 (Leonardo)	deal.II <code>~trilinos</code> , PETSc, MPICH	Missing Trilinos/Sacado required by LifeX 2.1
dealii/dealii Docker images (v9.7.1-jammy, master-jammy, v9.7.1-noble)	deal.II 9.7.1 with Trilinos, PETSc, p4est, VTK, HDF5	<code>boost_filesystemConfig.cmake</code> missing in the image
MOX mk container (deal.II 9.5.1)	deal.II 9.5.1, Boost 1.76, VTK, HDF5	API mismatch: LifeX 2.1 requires deal.II $\geq$ 9.7
<code>mk-dealiiimaster_latest.sif</code>	MK toolchain with deal.II master ($\geq$ 9.7), Boost, VTK, Trilinos, PETSc, p4est, HDF5	Adopted for the simulation campaign

Table 3: Container strategies evaluated for building LifeX on Leonardo.

The CMake configuration is invoked through `apptainer exec -cleanenv -bind`, which strips host environment contamination and mounts the workspace inside the container. Because the simulation campaign is purely two-dimensional, the build is configured with `-DLIFEX_DIM=2` and restricted to the two example targets actually used, `example_fluid_dynamics_airfoil` and `example_fluid_dynamics_external_flow`. A global `make all` cannot be used in 2D because some files in the test suite (e.g. `tests/multidomain_laplace/laplace.cpp`) rely on overloads of `GridTools::rotate` that exist only for three-dimensional triangulations, and the resulting compilation error stops the whole build.

The complete build script is reported in Listing 3.

```

1 #!/bin/bash
2 #SBATCH -A EUHPC_D35_025
3 #SBATCH -p dcgp_usr_prod
4 #SBATCH --time=00:24:00
5 #SBATCH -N 1
6 #SBATCH --ntasks-per-node=112
7 #SBATCH --job-name=lifex_build
8
9 set -euo pipefail
10
11 WORK_DIR="$HOME/Ice_acceleration_model_on_airplane_wing"
12 SIF="${WORK_DIR}/containers/mk-dealiiimaster_latest.sif"
13 SRC_SUBDIR="external/lifex-public"
14 BUILD_SUBDIR="external/lifex-public/build_2d"
15
16 rm -rf "${WORK_DIR}/${BUILD_SUBDIR}"
17
18 singularity exec --cleanenv \
19   --bind "${WORK_DIR}:/work" \
20   --bind /dev/shm:/dev/shm \
21   "${SIF}" bash -c "
22     source /u/sw/etc/profile

```



```

23 module load gcc-glibc dealii vtk
24
25 cmake -B /work/${BUILD_SUBDIR} -S /work/${SRC_SUBDIR} \
26     -DCMAKE_BUILD_TYPE=Release \
27     -DLIFEX_DIM=2 \
28     -DDEAL_II_DIR=${mkDealiiPrefix} \
29     -DLin_ALG=Trilinos
30
31 cmake --build /work/${BUILD_SUBDIR} \
32     --target example_fluid_dynamics_airfoil \
33         example_fluid_dynamics_external_flow \
34     -j 112
35 "

```

Listing 3: SLURM build script for LifeX on Leonardo (`job_build_leonardo.sh`). The Apptainer image is invoked through the legacy `singularity` command alias.

2.2.3. Mesh generation and pre-processing pipeline

The computational meshes are generated with Gmsh from parametric `.geo` scripts, driven by a set of Python generators (`generate_geo*.py`, `naca_generator_v*.py`) that expose the airfoil profile (NACA 0006 through NACA 0018), the domain extent, and the boundary-layer structure as input parameters.

The resulting mesh is hybrid: a structured layer of quadrilaterals wraps the airfoil surface to resolve the boundary layer, while the outer region is filled with unstructured triangles. Boundary entities follow a fixed tagging convention (1 = inlet, 2 = outlet, 3 = airfoil wall) matching the tags expected by the LifeX configuration (Section 2.1), and the mesh is exported in the MSH 2.2 format read natively by `deal.II`.

In accordance with the wall-resolved turbulence modeling approach described in Section 2.1.3, the near-wall region is carefully refined. The height of the first cell layer is adjusted until the estimated dimensionless wall distance satisfies the criterion $y^+ \lesssim 1$ at a Reynolds number of $Re = 10^6$. This ensures that the use of a homogeneous Dirichlet condition for $\bar{v}$ is physically valid.

Because a single malformed mesh wastes hours of allocated CPU time, every mesh is screened by a pre-processing pipeline before submission. Four independent Python checks are run in sequence:

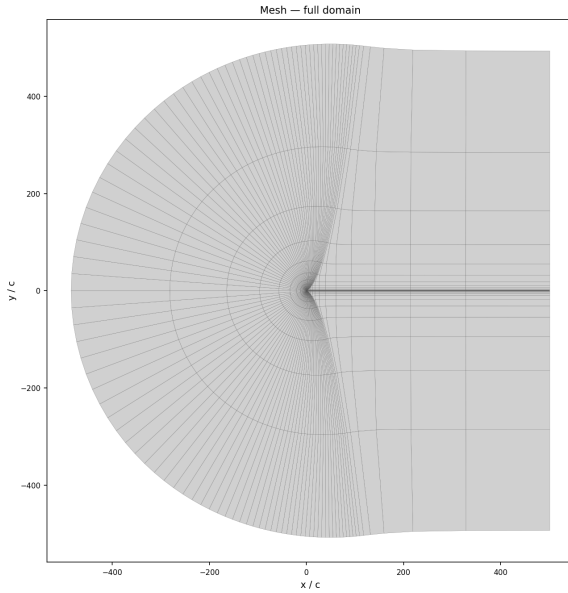
- `mesh_info.py` reports the node and element counts grouped by element type and physical tag and verifies that the inlet, outlet, and airfoil tags are all present;
- `check_extents.py` computes the bounding box of each tagged boundary, confirming that the inlet, outlet and airfoil entities are positioned where the geometry expects them;
- `check_jacobians.py` reads the mesh through `meshio` and computes the signed area of every quadrilateral via the cross product of its diagonals, flagging inverted or non-convex cells that would produce a negative Jacobian;
- `check_yplus.py` estimates the wall-normal y^+ at the first node by measuring the height of the cells adjacent to the airfoil edges and applying a flat-plate skin-friction correlation ($u_\tau/U_\infty \approx 0.036$), consistent with the estimate in Section 2.1.3.

Any mesh that fails one or more of these checks is automatically discarded and regenerated. An example of a mesh that has passed this quality control pipeline, used for the NACA 0012 simulations, is shown in Figure 2.

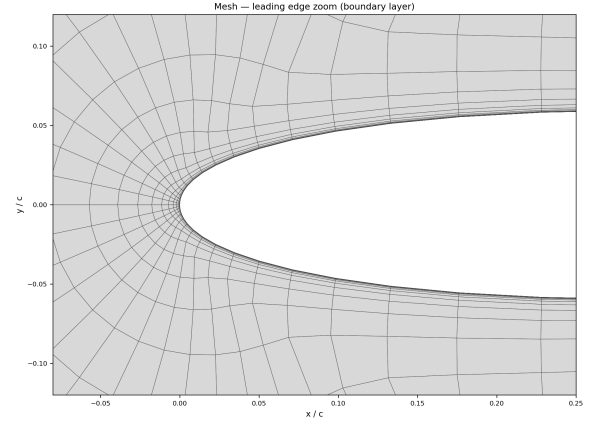
2.2.4. Simulation execution and post-processing

Each simulation is launched as a single-node SLURM job that runs the LifeX executable inside the Apptainer container with `mpirun -n 8`, distributing the mesh across eight MPI ranks on the `dcgp_usr_prod` partition.

The workspace is bind-mounted into the container and the run-time configuration is supplied as a JSON file, following the parameter structure described in Section 2.1: the turbulence model is activated through "Turbulence



(a) Overall domain



(b) Leading edge detail

Figure 2: Computational mesh for the NACA 0012 airfoil. (a) Overall domain, with a structured quadrilateral boundary layer around the profile and an unstructured triangular outer region. (b) Detail of the wall-resolved boundary layer at the leading edge, refined to give $y^+ \lesssim 1$ at $\text{Re} = 10^6$.

model": "Spalart-Allmaras", and the Reynolds number is set implicitly through the kinematic viscosity ($\nu = 10^{-6}$ with $U_\infty = 1$ and chord $c = 1$, giving $\text{Re} = 10^6$). Before execution, the configuration file is copied into the run directory `output/run_${SLURM_JOB_ID}/` as `config_used.json`, so that every result remains paired with the exact parameters that produced it.

During the run, the solver generates two primary forms of output: a time history of the aerodynamic coefficients, which is appended to `force.csv`, and the full velocity, pressure, and turbulent-viscosity fields, saved as HDF5 files with corresponding XDMF descriptors for visualization in ParaView. The complete SLURM submission script orchestrating this process is reported in Listing 5.

```

1  #!/bin/bash
2  #SBATCH -A EUHPC_D35_025
3  #SBATCH -p dcgp_usr_prod
4  #SBATCH --time=24:00:00
5  #SBATCH -N 1
6  #SBATCH --ntasks-per-node=8
7  #SBATCH --job-name=lifex_run
8  #SBATCH -o output/run_%j/solver.log
9
10 set -euo pipefail
11
12 WORK_DIR="$HOME/Ice_acceleration_model_on_airplane_wing"
13 SIF="${WORK_DIR}/containers/mk-dealii-master_latest.sif"
14 EXEC="/work/external/lifex-public/build_2d/examples/fluid_dynamics_airfoil
15     /lifex_example_fluid_dynamics_airfoil"
16 CONFIG="/work/cases/naca0012/config_new_method.json"
17 RUN_DIR="${WORK_DIR}/output/run_${SLURM_JOB_ID}"
18
19 mkdir -p "${RUN_DIR}"
20 cp "${WORK_DIR}/cases/naca0012/config_new_method.json" "${RUN_DIR}/
21     config_used.json"
22
23 singularity exec --cleanenv \
24     --bind "${WORK_DIR}:/work" \

```

```

24 --bind /dev/shm:/dev/shm \
25 "${SIF}" bash -c "
26     source /u/sw/etc/profile
27     module load gcc-glibc dealii vtk
28     export LD_LIBRARY_PATH=/work/external/lifex-public/build_2d/lib:\
        $LD_LIBRARY_PATH
29
30     cd /work/output/run_${SLURM_JOB_ID}
31     mpirun -n 8 ${EXEC} ${CONFIG}
32 "

```

Listing 4: SLURM run script for a single simulation (job_run_leonardo.sh).

The field output is inspected in ParaView, while the coefficient time histories are plotted with the `plot_forces.py` script (matplotlib with the `Agg` backend, so that figures are rendered without a display on the compute nodes).

Figures 3 and 4b show representative output for the NACA 0012 at $\alpha = 4^\circ$ and $\text{Re} = 10^6$. The pressure coefficient field (Figure 3) exhibits the suction peak on the upper surface near the leading edge that generates the net lift; the velocity field (Figure 4a) shows the flow accelerating over the suction side and the thin viscous wake downstream of the trailing edge; the spanwise vorticity (Figure 4b) isolates the shed boundary layers in the wake.

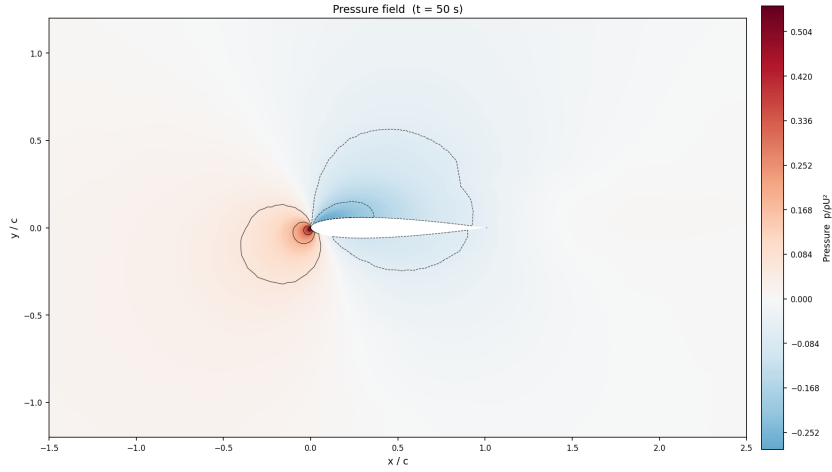
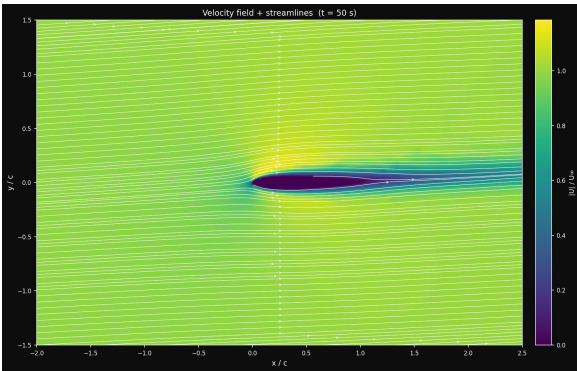
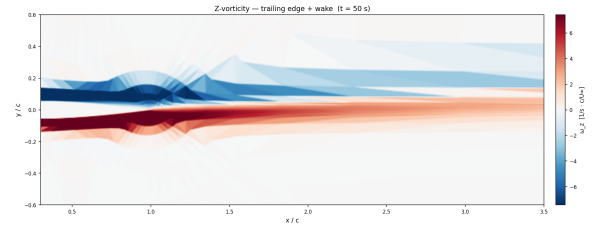


Figure 3: Pressure coefficient field around the NACA 0012 at $\alpha = 4^\circ$, $\text{Re} = 10^6$. The suction peak on the upper surface near the leading edge and the stagnation region on the lower side generate the net lift.



(a) Velocity magnitude with superimposed streamlines.



(b) Spanwise vorticity in the wake.

Figure 4: Flow field around the NACA 0012 at $\alpha = 4^\circ$, $\text{Re} = 10^6$. (a) The flow accelerates over the suction side and produces a thin viscous wake downstream of the trailing edge. (b) The spanwise vorticity isolates the shed boundary layers.

2.2.5. Dataset generation for the surrogate model

The simulation is not an end in itself: its role is to build the training set for the convolutional surrogate model presented in Section 3.

To build a comprehensive dataset, the parameter space was systematically explored. For each airfoil profile in a family of symmetric NACA sections (NACA 0006 to NACA 0018), the angle of attack was swept over the range $\alpha \in \{-14^\circ, -12^\circ, \dots, +14^\circ\}$ using templated configuration files (`config_isa_aoa_*.json`).

For each configuration, the pipeline produces a single label: the converged lift coefficient, C_L . These labels populate the `Y_cl` array of the final dataset, which is stored in the compressed NumPy format (`.npz`). The corresponding inputs are the Signed Distance Function representation of each geometry and the scalar pair of physical parameters (Re, α).

Computing the lift labels with the turbulence-resolved solver, rather than with a low-fidelity panel method, is what allows the surrogate to be trained against high-fidelity data in the high-incidence regime where thin-airfoil theory no longer holds.

The same workflow extends to the iced-airfoil geometries. A modified leading edge produced by an icing model could be created in future workflows and processed through the identical execution and post-processing chain, so that the dataset can later be augmented with iced profiles without any change to the simulation pipeline. The complete workflow is summarized in Figure 5.

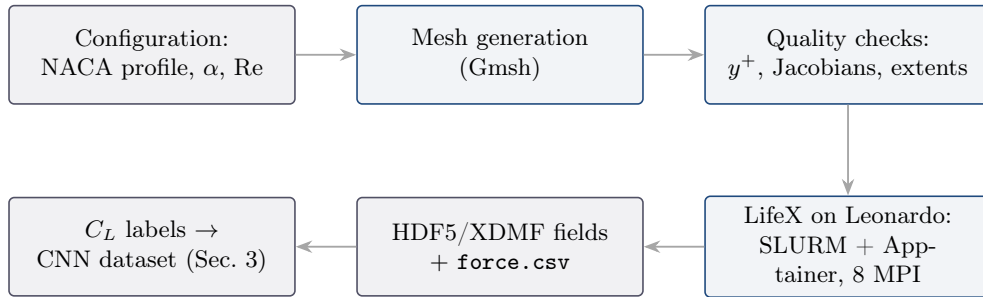


Figure 5: End-to-end simulation pipeline (top row left to right, then bottom row right to left). Mesh generation and quality control run interactively on the login node; the solver runs as a SLURM batch job inside the Apptainer container on the compute nodes; the resulting lift coefficients feed the convolutional surrogate model of Section 3.

3. Convolutional Neural Network

Building upon the dataset established in the preceding Section 2, this Section introduces a surrogate model designed to approximate the results of CFD simulations with a significant reduction in computational time.

Our approach is centered on a **Convolutional Neural Network (CNN)**, a deep learning architecture adept at learning the complex physical relationships that govern aerodynamic performance directly from simulation data.

Specifically, the network is designed to predict the lift coefficient (C_L) as a function of three key inputs:

1. the airfoil shape, represented as a **Signed Distance Function**;
2. the angle of attack (α);
3. the Reynolds number (Re).

By training on the generated simulation results, the CNN effectively learns to emulate the fluid dynamics solver, enabling near-instantaneous predictions.

The following sections will provide a comprehensive overview of this CNN model, from its underlying mathematical principles to its custom implementation designed for high-performance computing.

3.1. Mathematical Background

This section details the mathematical foundations of our surrogate modeling approach.

We first introduce the **Signed Distance Function (SDF)**, which transforms the airfoil’s complex geometry into a structured grid suitable for a neural network. We then provide a comprehensive breakdown of the **Convolutional Neural Network architecture**, explaining the mathematical operations within each layer and the physics-informed training algorithm used to learn the mapping from geometry and flow conditions to the aerodynamic lift coefficient.

3.1.1. Geometric Representation with Signed Distance Function

To construct a deep learning model capable of predicting aerodynamic coefficients, we need a robust method for representing the geometry of the airfoil $\Omega \subset \mathbb{R}^2$ and its boundary $\partial\Omega$ in a format suitable for Convolutional Neural Networks. Traditional boundary representations, such as coordinates of boundary points, are challenging to process with standard convolutional operators because they lack fixed spatial grids. To address this limitation, we represent the airfoil geometry using a **Signed Distance Function**, obtaining for each airfoil the result shown in Figure 6.



Figure 6: The Signed Distance Field representation of a NACA 0012 airfoil, discretized on a uniform grid. Negative values (blue) correspond to the airfoil interior, positive values (red) to the exterior while the boundary is represented by the zero-level contour (black line).

The SDF $\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}$ maps any point $P = (x, z)$ in the spatial domain to its signed distance from the boundary $\partial\Omega$:

$$\Phi(P) = \begin{cases} -d(P, \partial\Omega) & \text{if } P \in \Omega \\ d(P, \partial\Omega) & \text{if } P \notin \Omega \end{cases} \quad (10)$$

where $d(P, \partial\Omega)$ represents the Euclidean distance from the point P to the boundary $\partial\Omega$:

$$d(P, \partial\Omega) = \inf_{Q \in \partial\Omega} \|P - Q\|_2 \quad (11)$$

Using this definition, the boundary of the airfoil is implicitly defined as the zero-level set of the function:

$$\partial\Omega = \{P \in \mathbb{R}^2 \mid \Phi(P) = 0\} \quad (12)$$

while the interior corresponds to negative values ($\Phi(P) < 0$) and the exterior to positive values ($\Phi(P) > 0$).

In our numerical implementation, the boundary $\partial\Omega$ is defined as a closed polygon composed of an ordered set of n vertices $\{V_0, V_1, \dots, V_{n-1}\}$.

To guarantee scale invariance, the coordinates of the airfoil profile are first normalized by the chord length c , defined as the Euclidean distance between the leading edge point $(x_{\min}, z|_{x_{\min}})$ and the trailing edge point $(x_{\max}, z|_{x_{\max}})$:

$$c = \sqrt{(x_{\max} - x_{\min})^2 + (z_{\text{at } x_{\max}} - z_{\text{at } x_{\min}})^2} \quad (13)$$

where $x_{\max}$, $x_{\min}$, $z_{\max}$, and $z_{\min}$ are the extreme coordinates computed over the boundary vertices. Each vertex is then scaled by dividing its coordinates by c :

$$V_i = \left(\frac{x_{V_i}}{c}, \frac{z_{V_i}}{c} \right) \quad (14)$$

The boundary $\partial\Omega$ is then represented as the union of n linear segments $S_i = [V_i, V_{i+1}]$ (with $V_n = V_0$). The minimum distance from a grid point P to the boundary $\partial\Omega$ is computed by finding the minimum distance to each individual segment S_i :

$$d(P, \partial\Omega) = \min_{i=0, \dots, n-1} d(P, S_i) \quad (15)$$

For a given segment S_i defined by the endpoints $A = V_i$ and $B = V_{i+1}$, any point on the infinite line passing through A and B can be parameterized as $P(t) = A + t(B - A)$ for $t \in \mathbb{R}$. The orthogonal projection of the point P onto this line is obtained by minimizing the distance $\|P - P(t)\|^2$, which yields the projection parameter:

$$t^* = \frac{(P - A) \cdot (B - A)}{\|B - A\|_2^2} \quad (16)$$

To restrict the projection to the finite line segment S_i , t^* is clamped to the interval $[0, 1]$:

$$t = \max(0, \min(1, t^*)) \quad (17)$$

The projection point on the segment is then $P_{\text{proj}} = A + t(B - A)$, and the distance to the segment is:

$$d(P, S_i) = \|P - P_{\text{proj}}\|_2 \quad (18)$$

To determine the sign of the distance field, we implement the **Winding Number Algorithm** to check whether a grid point P lies inside or outside the closed boundary. The winding number $wn(P)$ computes the number of times the boundary curve wraps around the point P . For a closed polygon, it is computed by casting a horizontal ray starting from P and pointing in the positive x -direction, and counting the signed crossings of this ray with the polygon edges:

- an **upward crossing** occurs if an edge $S_i = [V_i, V_{i+1}]$ crosses the horizontal ray from below, i.e., $V_{i,z} \leq z_P < V_{i+1,z}$ and P lies to the left of the directed line segment;
- a **downward crossing** occurs if an edge S_i crosses the horizontal ray from above, i.e., $V_{i+1,z} \leq z_P < V_{i,z}$ and P lies to the right of the directed line segment.

The relative position of the point P with respect to the directed segment $[A, B]$ is determined mathematically using the 2D cross product:

$$\text{cross}(P, A, B) = (A_x - x_P)(B_z - z_P) - (B_x - x_P)(A_z - z_P) \quad (19)$$

If $\text{cross}(P, A, B) > 0$, the point P lies to the left of the directed segment, whereas if $\text{cross}(P, A, B) < 0$, it lies to the right. The winding number is incremented by 1 for every upward crossing and decremented by 1 for every downward crossing. A point P is strictly inside the domain Ω if and only if the total winding number is non-zero:

$$P \in \Omega \iff wn(P) \neq 0 \quad (20)$$

Once the signed distance is computed for any point, we project it onto a discrete uniform grid. The spatial domain is discretized into a Cartesian grid of size $N \times N = 150 \times 150$ over the bounding box $[X_{\min}, X_{\max}] \times [Z_{\min}, Z_{\max}]$, with:

$$X_{\min} = -0.15, \quad X_{\max} = 1.01, \quad Z_{\min} = -0.12, \quad Z_{\max} = 0.12 \quad (21)$$

The grid spacing is defined by:

$$\Delta x = \frac{X_{\max} - X_{\min}}{N - 1}, \quad \Delta z = \frac{Z_{\max} - Z_{\min}}{N - 1} \quad (22)$$

For each grid node indexed by (row, col) for $row, col = 0, \dots, N - 1$, the corresponding spatial coordinates are given by:

$$x_{col} = X_{\min} + col \cdot \Delta x, \quad z_{row} = Z_{\max} - row \cdot \Delta z \quad (23)$$

This discretized representation yields a 2D matrix of shape 150×150 , where each entry contains the signed distance value. This matrix provides a spatially structured representation of the geometry that preserves boundary details, making it perfectly suited as an input to the convolutional branch of the neural network.

3.1.2. Convolutional Neural Networks for Aerodynamic Prediction

The architecture of our **Convolutional Neural Network** is designed as a multi-input model to effectively process both the spatial geometric data and the scalar physical parameters. A high-level schematic of this design, inspired by the model presented by Cuozzo et al. (2026), is depicted in Figure 7.

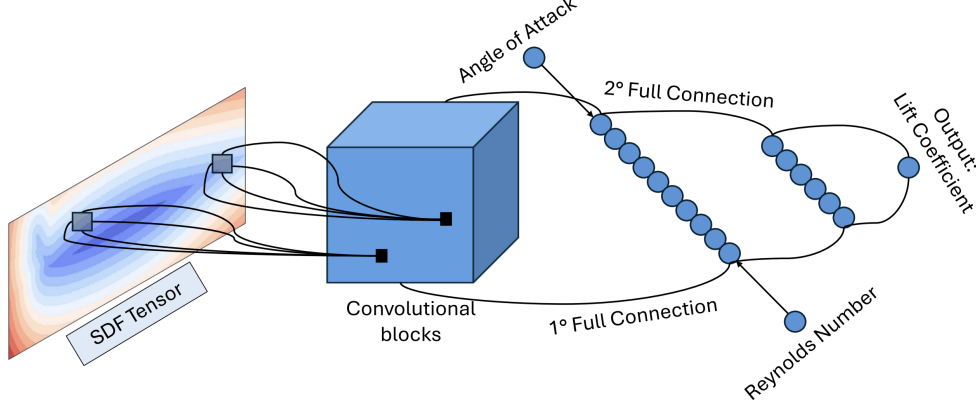


Figure 7: Architecture of the Convolutional Neural Network used for aerodynamic prediction.

The network is logically divided into three main components:

1. a convolutional feature extractor, responsible for interpreting the airfoil geometry. It accepts the 2D SDF matrix as input and processes it through a series of **Conv2DLayers**, each followed by a **LeakyReLU** activation. This stack of layers hierarchically extracts relevant spatial features, such as leading-edge curvature and surface details. The resulting multi-channel feature maps are then transformed into a one-dimensional vector by a **FlattenLayer**, producing a learned representation of the airfoil's shape;
2. a feature fusion stage, in which, the flattened geometric feature vector from the convolutional branch is concatenated with the scalar physical parameters (the angle of attack and the Reynolds number) tensor using a **ConcatenateLayer**. This creates a single feature vector that contains comprehensive information about both the airfoil's shape and the current flow conditions;
3. finally the unified vector is passed to the regression head, which consists of a sequence of **DenseLayers**. This fully-connected network maps the combined high-level features to a single scalar output, which represents the predicted lift coefficient, C_L .

Convolutional Layer

The **Convolutional Layer** (**Conv2DLayer**) is an operator specifically designed to process grid-like data such as the 2D SDF. Its primary function is to detect local spatial patterns within the input by applying a set of learnable filters. The difference with the dense layer, which treats all input pixels independently, is that a convolutional layer preserves the spatial relationship between pixels by operating on localized neighborhoods.

At its core, the operation is a discrete 2D convolution, which can be understood as a *sliding window dot product*. A small matrix of learnable weights, known as the kernel (K), slides over the input feature map (I). At each position, an element-wise multiplication between the kernel and the overlapping region of the input is performed, and the results are summed up to produce a single pixel in the output feature map (O).

For a single-channel input I of size $H \times W$ and a kernel of size $F \times F$, the value of the output O at position (i, j) is given by:

$$O(i, j) = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} I(i-m, j-n) \cdot K(m, n) + b \quad (24)$$

where b is a learnable scalar bias term applied to the entire output feature map.

The key principle here is parameter sharing: the *same* kernel K and bias b are used across all positions of the input, allowing the network to detect the same feature regardless of where it appears in the SDF².

In deep networks, layers typically process multi-channel inputs. If the input feature map has C_{in} channels³, the kernel must be three-dimensional, with a size of $F \times F \times C_{in}$. The convolution is then performed across all channels simultaneously. The results from each channel are then summed together to produce a single value for the 2D output map.

To generate an output with C_{out} feature maps, we simply apply C_{out} independent kernels. Each kernel is responsible for learning a different spatial feature, allowing the layer to build a complete representation of the input geometry.

The behavior of the convolution is controlled by two hyperparameters:

- the stride (S) defines the step size the kernel matrix as it traverses the input grid. While a unit stride ($S = 1$) evaluates adjacent localized neighborhoods sequentially, a stride of $S > 1$ skips spatial coordinates, downsampling the spatial resolution of the resulting feature map;
- padding (P)⁴ consists in appending a perimeter of zero-valued elements around the input grid boundaries. This can be used to control the output dimensions and mitigate the loss of information at the edges.

The dimensions of the output feature map ($H_{out} \times W_{out}$) are determined by the input dimensions ($H_{in} \times W_{in}$), the kernel size (F), the stride (S) and the padding (P) using the following formula:

$$W_{out} = \frac{W_{in} - F + 2P}{S} + 1 \quad \text{and} \quad H_{out} = \frac{H_{in} - F + 2P}{S} + 1 \quad (25)$$

From an implementation perspective, this operation is typically realized as a set of nested loops that iterate over each position in the output map, each element of the kernel, and each input channel, performing the required multiply-accumulate operations⁵.

Non-Linear Activation (Leaky ReLU)

After each `Conv2DLayer` or `DenseLayer` performs its linear operation, an activation function applies a non-linear transformation element-wise to its output. Without this, a deep neural network, regardless of its number of layers, would behave as a single, large linear model. The introduction of non-linearity is what gives the network its power to approximate arbitrarily complex, non-linear relationships, such as the one between airfoil geometry and lift coefficient.

In this project, a **Leaky Rectified Linear Unit** (Leaky ReLU) is implemented as the `LeakyReLULayer`. This function, shown in Figure 8, is a popular and effective choice in modern deep learning due to its computational efficiency and its ability to mitigate some of the common problems encountered during training.

The Leaky ReLU function is defined mathematically as:

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \quad (26)$$

where α is a small, positive constant (a hyperparameter, typically set to a value like 0.01).

The function behaves as a simple identity for all positive input values. For negative input values, instead of mapping them to zero like the standard ReLU function, it allows a small, non-zero gradient to pass through.

The primary advantage of the Leaky ReLU over the standard ReLU is its handling of negative inputs. In a standard ReLU, if a neuron's input is consistently negative, its gradient will always be zero. This means the

²In more details, if the kernel learns to recognize a leading edge curve of a profile, it will successfully detect the curve whether it appears at the top of the grid or in a bottom corner.

³This is not our case, since in the SDF we have only one input channel ($C_{in} = 1$).

⁴In our implementation, padding is set to 0.

⁵We will explain in a better way highlighting our C++ implementation in Section 3.2.3 regarding our implementation in C++

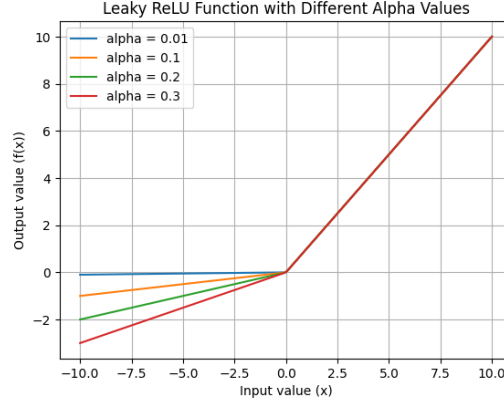


Figure 8: Example of Leaky ReLU function with different α .

neuron's weights will never be updated, and it can effectively "die", ceasing to contribute to the network's learning process. The Leaky ReLU prevents this "dying ReLU" problem by ensuring that a small gradient always exists, even for negative inputs.

From the backpropagation algorithm, the gradient of the Leaky ReLU is simple and computationally inexpensive to calculate:

$$\frac{d}{dx} \text{LeakyReLU}(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{if } x \leq 0 \end{cases} \quad (27)$$

During the backward pass of training, this straightforward derivative is used to scale the incoming gradient from the subsequent layer:

- if the input to the activation function was positive during the forward pass, the gradient passes through unchanged;
- if it was negative, the gradient is scaled down by the factor α .

This simple conditional check makes the backward pass for the `LeakyReLU` layer extremely fast, contributing to the overall efficiency of the training process.

Structural Layers

After the convolutional network has extracted a hierarchy of spatial features, the resulting multi-channel feature maps must be prepared for integration with the scalar parameters and for final processing by the regression head. This preparation is handled by two complementary **structural layers**, the `FlattenLayer` and the `ConcatenateLayer`, that are parameter-free: they do not contain any learnable weights or biases since their sole purpose is to rearrange data tensors.

Flatten Layer

The `FlattenLayer` links the convolutional layers and the fully-connected layers. Convolutional layers output a 4D tensor with a shape of `[Batch Size, Channels, Height, Width]`, preserving spatial information. However, dense layers require their input to be a 2D tensor of shape `[Batch Size, Features]`, where each row is a one-dimensional vector representing a single sample.

The flatten operation reshapes an input tensor T_{in} with dimensions $[B \times C_{out} \times H_{out} \times W_{out}]$ into an output tensor T_{out} with dimensions $[B \times D]$, where the new feature dimension D is the product of the input's channel, height and width dimensions:

$$D = C_{out} \times H_{out} \times W_{out} \quad (28)$$

During the forward pass, this is a straightforward memory rearrangement.

During the backward pass for backpropagation, the `FlattenLayer` performs the inverse operation: it receives an incoming gradient tensor of shape $[B \times D]$ and reshapes it back to the original 4D shape $[B \times C_{out} \times H_{out} \times W_{out}]$. This ensures that the gradients are correctly propagated back to the preceding convolutional layer in the required spatial format.

Concatenate Layer

Once the geometric features have been flattened into a vector, they must be combined with the scalar physical parameters by the **ConcatenateLayer**, that takes multiple input tensors and joins them together along a specified axis⁶.

Mathematically, given the flattened geometric feature tensor T_{flat} of shape $[B \times D_1]$ and the scalar input tensor T_{scalar} (containing angle of attack and Reynolds number) of shape $[B \times D_2]$, the Concatenate Layer produces an output tensor T_{concat} of shape $[B \times (D_1 + D_2)]$. This operation creates a unified feature vector that encodes both geometric and physical information for the network to make a prediction.

During backpropagation, the process is reversed. The incoming gradient tensor is split according to the original input sizes:

- the first D_1 elements of the gradient vector are passed back to the **FlattenLayer**'s branch;
- the D_2 elements corresponding to the scalar inputs are also computed, although they terminate at the input layer.

After this fusion step, the resulting unified tensor is ready to be processed by the final regression head of the network.

Dense Layer and final regression

The final computational stage of the network is handled by the **Dense Layer** (**DenseLayer**), known as a fully-connected layer. Its role is to learn the final mapping from the feature space to the target scalar value (the lift coefficient C_L).

A dense layer applies a linear transformation to its input vector. For an input vector $\mathbf{x} \in \mathbb{R}^{D_{in}}$, the layer computes an output vector $\mathbf{y} \in \mathbb{R}^{D_{out}}$ using a learnable weight matrix $W \in \mathbb{R}^{D_{out} \times D_{in}}$ and a learnable bias vector $\mathbf{b} \in \mathbb{R}^{D_{out}}$. The operation is defined as:

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \quad (29)$$

This operation performs a weighted sum of all input features to compute each output feature. Every neuron in the input layer is connected to every neuron in the output layer, allowing the network to model complex interactions between all elements of the input feature vector.

This layer is used in the regression head, which consists of a sequence of **DenseLayers**, each followed by a **LeakyReLU** activation:

- the initial dense layers take concatenated feature vector as input. Their purpose is to find and amplify the non-linear correlations between the learned geometric features from the SDF and the raw physical parameters (α and Re);
- the last layer is configured to have a single output neuron ($D_{out} = 1$) and does not have a subsequent activation function. This single neuron's output directly corresponds to the network's prediction for the lift coefficient (C_L).

During the backward pass, the dense layer must compute three distinct gradients based on the incoming gradient from the subsequent layer $\frac{\partial L}{\partial \mathbf{y}}$, where L is the loss:

- the gradient with respect to the weights (W) is calculated as the outer product of the incoming gradient and the layer's input vector from the forward pass:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \mathbf{y}} \cdot \underbrace{\mathbf{x}^T}_{\frac{\partial \mathbf{y}}{\partial W}} \quad (30)$$

- the gradient with respect to the biases ($\mathbf{b}$) is equal to the incoming gradient:

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}} \quad (31)$$

⁶In our case, this is the feature axis (axis 1)

- the gradient with respect to the input (x) must be passed back to the preceding layer. It is computed by multiplying the transposed weight matrix by the incoming gradient:

$$\frac{\partial L}{\partial \mathbf{x}} = W^T \cdot \frac{\partial L}{\partial \mathbf{y}} \quad (32)$$

These calculations are the fundamental operations that enable the network's regression head to learn from the training data.

Network Training

The process of training the neural network involves iteratively adjusting its parameters to minimize a loss function, which quantifies the discrepancy between the model's predictions and the desired outcomes. Our approach embeds physical knowledge directly into the training process, making our model a type of **Physics-Informed Neural Network (PINN)**. This is achieved through a composite loss function that balances empirical data accuracy with a theoretical physical constraint.

The total loss, L , is a weighted sum of data-driven term and a physics-informed term:

$$L = L_{\text{data}} + \lambda L_{\text{physics}} \quad (33)$$

where λ is a hyperparameter that controls the strength of the physical regularization.

The first component is the standard **Mean Squared Error (MSE)**, which measures the difference between the network's predicted lift coefficients (P_i) and the ground-truth values from the CFD simulations (T_i):

$$L_{\text{data}} = \frac{1}{N} \sum_{i=1}^N (P_i - T_i)^2 \quad (34)$$

This term ensures the network learns to faithfully replicate the high-fidelity simulation data generated for this work.

The second component acts as a physical constraint, guiding the network's predictions to be consistent with established aerodynamic theory. This term is based on **Thin-Airfoil Theory**, which provides an analytical approximation for the lift coefficient at small angles of attack. The general form of this relationship is $C_L = 2\pi(\alpha - \alpha_{L=0})$, where $\alpha_{L=0}$ is the zero-lift angle of attack.

For symmetric airfoils, such as those in our dataset, the zero-lift angle of attack is, by definition, $\alpha_{L=0} = 0$. This simplifies the theoretical relationship to the linear form:

$$C_L \approx 2\pi\alpha \quad (35)$$

The physical loss in the network leverages this simplified form, penalizing deviations from this theoretical relationship:

$$L_{\text{physics}} = \frac{1}{N} \sum_{i=1}^N M_i \cdot (P_i - 2\pi\alpha_i)^2 \quad (36)$$

where M_i is a mask function, introduced because Thin-Airfoil Theory is only valid in the linear flight regime. Therefore, the mask is defined as:

$$M_i = \begin{cases} 1 & \text{if } |\alpha_i| \leq 10^\circ \\ 0 & \text{if } |\alpha_i| > 10^\circ \end{cases} \quad (37)$$

This ensures the physical constraint is only active at low angles of attack.

In the non-linear, high-angle-of-attack regime, the mask deactivates this term ($M_i = 0$), forcing the network to rely entirely on the empirical CFD data, which accurately captures the complex flow separation phenomena.

By explicitly building in the assumption of symmetry, this physics-informed regularizer provides a robust theoretical anchor for the network's predictions without being applied in regimes where its underlying assumptions would be violated.

20

In the following sections, we decide to highlight the most interesting implementations of these components, detailing key choices and memory optimization layouts, as well as providing an explanation of the core functions implemented.

3.2.1. Geometric Preprocessing: SDF Generator

The generation of the Signed Distance Function is encapsulated within the `SDFGenerator` class, which maps airfoil geometries to structured 2D tensors suitable for convolutional processing.

To maximize CPU cache efficiency and minimize cache misses during spatial calculations, boundary vertices are stored in a `std::vector<Point>`, where `Point` is a simple structure holding `x` and `z` coordinates. This choice creates a single contiguous array for the boundary data.

Upon initialization, the constructor ensures scale invariance (as detailed in Section 3.1.1) by calling two private helper methods: `computeCord()` calculates the chord length c using `std::hypot` to prevent numerical overflow⁷ and `normalizePoint()` scales all vertex coordinates accordingly⁸.

With a normalized boundary in place, the `SDFGenerator` computes the signed distance for each point on the target grid. This is achieved through a set of optimized geometric kernel functions:

- `computeDistance(...)` `const` calculates the minimum Euclidean distance from a grid point to a boundary segment (as it is shown in Equations (16) and (17)). It includes a safety check for degenerate edges (where the two end-point of the segment coincide) to prevent division-by-zero errors;
- `getMinDistance(...)` `const` iterates through all boundary edges of the airfoil, calling for each one `computeDistance` and returning the absolute minimum distance found. Additionally, we used `static_cast<int>(airfoil.boundary.size())` to safely convert the unsigned vector size to a signed integer, preventing signed/unsigned comparison warnings;
- finally, the `windingAlgorithm(...)` `const` implements the Winding Number Algorithm previously explained (Section 3.1.1) to verify whether a grid point p lies inside or outside the closed boundary. This is computed by analyzing edge crossings and using the 2D cross product (Equation (19))⁹, returning `true` if the point is inside the domain.

Once these values are computed for every grid point, the `mappingToMatrix` function converts the resulting 1D flattened SDF data into the required 150×150 2D matrix format and writes it to a file, utilizing the **RAII** (**R**esource **A**cquisition **I**s **I**nitialization) paradigm via `std::ofstream outfile(filename)` that automatically closes the file descriptor when `outfile` goes out of scope, in order to prevent system resource leaks in case of exceptions.

The most computationally intensive part of the process is executing these kernels for each one of the $150 \times 150 = 22,500$ points on the grid. This task is a classic example of an *embarrassingly* parallel problem and it is implemented in the `generateTensorMPI` method, the details of which are provided in Section 3.3.

3.2.2. Core abstractions and memory management

To achieve high computational throughput while preserving a flexible design, the system introduces two core abstractions: a unified multi-dimensional tensor representation and a polymorphic layer interface.

Tensor

The foundational data structure of our CNN implementation is the `Tensor` class, which manages both forward activation data (`_data`) and backward gradients (`_grad`) as multi-dimensional arrays of single-precision floating-point numbers.

⁷This function scales intermediate values before performing the sum of squares, avoiding the premature overflow or underflow that can occur when computing the norm directly as $\sqrt{\Delta x^2 + \Delta z^2}$

⁸This normalization step will be valuable for future implementations aimed at studying the effect of ice accumulation on the lift coefficient.

⁹Implemented through the `crossProduct(Point p, Point a, Point b)` `const` with each point passed by value, in order improve performance for small `structs` by allowing them to be passed directly in CPU registers, avoiding memory indirection.

First of all, the constructor `Tensor(std::vector<size_t> shape)` initializes the shape descriptor and sets the `_total_size` member through a member **initializer list**. It pre-allocates and zero-initializes the storage vectors using `std::vector::resize`, that guarantees contiguous memory allocation. Direct access to this underlying memory is provided via the `get_data()` and `get_grad()` methods, which return raw pointers (`float*`) using the `.data()` member function.

Beyond raw pointer access, the class provides safe, native array-like indexing for flattened 1D elements through overloaded `operator[]` methods, offering both mutable and read-only (`const`-qualified) references. Utility functions are also optimized: gradient values are efficiently reset before each backward pass using the `std::fill` algorithm within the `zero_grad()` method.

Furthermore, the total element count is computed by `calculate_size()`, which is declared as a `static` method¹⁰ and accepts a constant reference to avoid the overhead of copying the shape vector.

Finally, the class strictly adheres to `const` correctness, ensuring that utility methods like `print_metadata()` are executed safely without modifying the object's internal state.

Layer

The abstract base class `Layer`, defined in `Layer.hpp`, establishes a unified interface for all CNN components by leveraging runtime polymorphism to decouple the high-level network topology from the specific implementations of individual layers.

Runtime polymorphism is achieved through virtual functions and dynamic dispatch. When a class declares virtual methods, the compiler generates a virtual method table (`vtable`) containing pointers to these functions. Each object instance is implicitly equipped with a pointer (`vptr`) to its class's `vtable`. During execution, when a virtual method is called via a base class pointer, the program resolves the call dynamically: it dereferences the object's `vptr` to access the `vtable` and executes the overridden method corresponding to the actual derived type.

This mechanism is fundamental to our design. Rather than relying on conditional logic to handle the different mathematical operations, polymorphism allows the model to store and manage all layers uniformly within a single container (`std::vector<std::shared_ptr<Layer>`). In addition, this design ensures high scalability and the effortless integration of new layer types simply by inheriting from the `Layer` base class.

Following the idea of implementing the polymorphism, we declared a virtual destructor `~Layer() = default`, ensuring that when a derived layer object is destroyed via a base class pointer, the derived class's destructor is executed, preventing resource leaks of dynamically allocated memory.

The core operations of the network are declared as pure virtual functions (indicated by the `= 0` suffix), which must be overridden by every concrete layer subclass. Specifically, we implemented:

- `forward`, that computes forward activations, and `backward`, that performs backpropagation, both accepting and returning `std::shared_ptr` in order to guarantee automatic memory deallocation when the tensors are no longer needed;
- `get_weights_and_grads()` and `get_biases_and_grads()` that returns `std::pair<float*, float*>` packaging the weight/bias arrays and their corresponding gradients into a single return value. Returning non-const pointers allows the optimizer to modify the parameter values directly in place.

Finally, architectural decoupling is achieved through a forward declaration of the `Optimizer` class; this resolves potential circular dependencies while allowing the `update_weights(std::shared_ptr<Optimizer>)` method to delegate parameter updates to external optimization algorithms seamlessly.

3.2.3. Computational and activation layers

The computational layers manage the network's mathematical transformations and contain all trainable parameters, while activation layers introduce the necessary non-linear activation boundaries.

¹⁰This allows the method to be invoked without a class instance, for example, within a member initializer list.

DenseLayer

The concrete `DenseLayer` class implements the fully-connected linear transformation $\mathbf{y} = W\mathbf{x} + \mathbf{b}$, where W is the weight matrix and $\mathbf{b}$ is the bias vector, as described in details in the Section 3.1.2.

First of all, the constructor `DenseLayer(int input_features, int output_features)` allocates the weight and bias tensors utilizing `std::make_shared` rather than direct initialization via `std::shared_ptr(new Tensor(...))`. While this last one forces two separate heap allocations¹¹, `std::make_shared` performs a **single contiguous memory allocation** for both the reference-counting control block and the managed `Tensor` object, in order to reduce heap fragmentation and improve CPU cache locality by keeping the control metadata and the tensor data adjacent in memory.

To ensure gradient stability at the start of training, the weights are initialized according to the **Xavier/Glorot scheme**, drawing from a uniform distribution bounded by $\pm\sqrt{6/(N_{\text{in}} + N_{\text{out}})}$ using `std::default_random_engine` and `std::uniform_real_distribution<float>`.

During the **forward** pass, the input tensor is explicitly cached (`_input_cache`) to satisfy the mathematical requirements of backpropagation, as the input vector $\mathbf{x}$ must be retained to compute the weight updates as $\frac{\partial L}{\partial W} = \mathbf{x} \otimes \frac{\partial L}{\partial \mathbf{y}}$.

Moreover, the linear transformation $Y = XW^T + \mathbf{b}$ is evaluated through three nested loops:

- the outermost loop iterates over the batch size b ;
- the middle loop iterates over the output features j ;
- the innermost loop iterates over the input features i to evaluate the dot product $Y_{b,j} = \sum_i X_{b,i}W_{j,i} + b_j$.

The weight matrix W is stored in a transposed format of shape $(N_{\text{out}}, N_{\text{in}})$. This ensures that as the innermost loop increments its index over the input features, it accesses both the input tensor X and the weight tensor W with a unit stride. This design maximizes spatial locality, reduces cache misses, and enables the compiler to generate efficient SIMD vector instructions.

To conclude, also the **backward** method computes the gradients through three distinct sets of nested loops:

1. the gradient with respect to the inputs is calculated as $dX_{b,i} = \sum_j dY_{b,j}W_{j,i}$. In this loop structure, the weight matrix is traversed column-wise, resulting in non-contiguous memory access;
2. the gradient with respect to the weights is accumulated as $dW_{j,i} = \sum_b dY_{b,j}X_{b,i}$ using loops over j , i , and the batch dimension b . The innermost loop iterates over b , calculating the outer product of the output gradients and cached inputs;
3. the gradient with respect to the biases is accumulated as $dB_j = \sum_b dY_{b,j}$ by summing the output gradients across all batch elements for each output feature.

LeakyReLULayer

The concrete `LeakyReLULayer` class implements homonymous element-wise activation function shown in Equation (26). The negative slope parameter α is configured into the constructor `LeakyReLULayer(float alpha = 0.05f)`.

The interesting thing in this code is the use of the ternary operator:

- in the **forward** method, the element-wise activation is evaluated inside a contiguous loop using raw pointers `out_ptr[i] = (in_ptr[i] > 0.0f) ? in_ptr[i] : _alpha * in_ptr[i];`
- in the **backward** method, that is used to evaluate the gradient with respect to the input by scaling the output gradient as shown in Equation (27), implemented as `din_ptr[i] = dout_ptr[i] * ((cache_ptr[i] > 0.0f) ? 1.0f : _alpha).`

Because the derivative of the activation function is conditioned on the sign of the original input activations, caching the forward input tensor inside `_input_cache` is necessary to retrieve the values during the backward pass.

To conclude this class, because the activation layer contains no learnable parameters, the overridden method `update_weights` is implemented as a no-op. Moreover, the parameter accessors `get_weights_and_grads()` and `get_biases_and_grads()` return a pair of type-safe null pointers, `{nullptr, nullptr}`.

¹¹One for the object and one for its control block.

Conv2DLayer

The concrete **Conv2DLayer** class extends the abstract **Layer** interface to perform 2D spatial convolutions on four-dimensional tensors, which logically represent **batch samples, channels, height and width**.

Upon instantiation, the constructor initializes the convolutional kernels using a **Gaussian Xavier scheme** to ensure variance stability across deep network layers. The standard deviation for this distribution is computed as $\sigma = \sqrt{2/(F_{\text{in}} + F_{\text{out}})}$, where the fan-in and fan-out dimensions are defined as $F_{\text{in}} = C_{\text{in}} \times K^2$ and $F_{\text{out}} = C_{\text{out}} \times K^2$, with K denoting the kernel size.

During the **forward** pass, the spatial dimensions of the output tensor are explicitly calculated using Equation (25) and the resulting tensor is allocated contiguously via `std::make_shared`. Because the system flattens all multi-dimensional tensors into 1D contiguous vectors for optimal memory layout, the layer utilizes a private inline helper function, `idx4`, to resolve 4D indices to 1D offsets. This enforces a strict row-major mapping:

$$\text{Offset}(a, b, c, d) = [(a \times \text{dim}_b + b) \times \text{dim}_c + c] \times \text{dim}_d + d \quad (42)$$

where a, b, c, d represent the target indices across the batch, channel, and spatial dimensions.

The forward convolution operation is executed through seven-nested loop. The outer four loops iterate over the batch elements n , output channels oc and spatial output coordinates oh and ow , while the inner three loops traverse the input channels ic and kernel dimensions kh and kw . To handle boundaries without incurring the overhead of allocating memory for physical zero-padding, the implementation uses implicit padding. Source coordinates are dynamically evaluated at each step as $ih = oh \times S + kh - P$ and $iw = ow \times S + kw - P$:

- if these fall outside the input boundaries, the multiplication is safely bypassed using a **continue** statement;
- otherwise, the flattened indices are retrieved via `idx4`, and the kernel-input products are accumulated into a running sum initialized with the appropriate channel bias.

Finally, the **backward** method evaluates gradients using two distinct loop structures:

- the first structure computes the parameter gradients: the bias gradient (dB) aggregates the output gradients directly, while the weight gradient (dW) accumulates the product of the output gradients and the cached forward inputs;
- the second structure computes the gradient with respect to the input (dX) by performing a full convolution between the output gradients and the flipped kernel filters. To achieve this, the loop accesses the weights using inverted spatial coordinates:

$$kh_{\text{orig}} = K - 1 - kh, \quad kw_{\text{orig}} = K - 1 - kw \quad (43)$$

3.2.4. Structural Layers

The **structural layers** are parameter-free components designed to rearrange and route data tensors across the branches of the network.

FlattenLayer

The **FlattenLayer** class extends the base **Layer** interface to perform a logical reshaping of the 4D feature maps (generated by the convolutional branch) into a 2D tensor suitable for the subsequent **ConcatenateLayer**.

The **forward** method calculates the flattened size $F = C \times H \times W$ and allocates a 2D output tensor of shape $[B \times F]$ via `std::make_shared`. Since the flatten operation is basically a reshape, the data is copied from the input to the output tensor sequentially in a single loop, preserving the element order across contiguous memory arrays.

The **backward** method receives the 2D gradient tensor `grad_output` (of shape $[B \times F]$), validates the batch size consistency, allocates a 4D gradient tensor matching the cached shape `_input_shape_cache`¹² and performs the inverse sequential copy `grad_in_ptr[i] = grad_out_ptr[i]`, reconstructing the 4D gradient layout for the preceding convolutional layer.

¹²This member attribute of type `std::vector<size_t>` preserves the original spatial dimensions ($C \times H \times W$) to reconstruct and propagate the gradient back.

ConcatenateLayer

The `ConcatenateLayer` class is used to obtain a single tensor that combines the information from the **Convolutional Layer** and the scalar values of the Reynolds number and angle of attack.

Beyond the mathematical formalism about the shape of the matrices described in a detailed way in Section 3.1.2, the merge operations are implemented with nested loops:

- the outer loop iterates over the batch size n , computing the row offsets for the first input ($n * \text{_features_first_cache}$), the second input ($n * \text{_features_second_cache}$) and the output ($n * (\text{_features_first_cache} + \text{_features_second_cache})$);
- two inner loops then copy the data, ensuring row-wise contiguous layout.

During the `backward` method, this operation is inverted. The layer receives a combined gradient tensor of shape $[B \times (D_1 + D_2)]$ and must route the gradients back to their respective origins, reversing the copy logic: for each batch element, it copies the first D_1 components of the output gradient to the first input gradient buffer and the remaining D_2 components to the second input gradient buffer. Both elements are then returned in a standard vector `{grad_first, grad_second}`.

3.2.5. Training pipeline, loss and optimizers

The network's optimization and training pipeline is coordinated by mathematical loss functions and state-tracking optimization solvers.

Loss

Unlike the object-oriented structure of the computational layers, the **physics-informed loss function** and its corresponding gradient are implemented as non-member utilities within the `Loss` namespace. This design choice avoids the overhead of object instantiation and maintains a functional style for stateless operations.

In the `forward` method¹³, the composite physics-informed loss, described in Section 3.1.2, is computed in a single loop using raw pointers retrieved via `get_data()` to ensure contiguous memory traversal. When computing the physics-informed penalty, the masking logic from Equation (37) is applied: the linear regime limit is saved in an anonymous namespace at the beginning of the file, in order to restrict the scope of this constant. The average data and physics terms are computed and normalized by the number of elements using `static_cast<float>(n)`

Finally, the `backward` method is implemented in a similar way.

Optimizer and Adam Optimizer

Similar to the layer architecture, the optimization logic is strictly decoupled from the neural network topology via an abstract `Optimizer` base class. This unified interface ensures that different numerical solvers can be swapped seamlessly, exposing a single pure virtual function (`apply_gradients(float* weights, float* grads, size_t size)`) to handle dynamic dispatch interface that must be implemented by concrete subclasses.

The only concrete implementation of this interface is the `AdamOptimizer`, which executes the **Adaptive Moment Estimation** algorithm detailed in Section 3.1.2.

To map specific weight and bias arrays to their corresponding optimization states without coupling the layers to the optimizer, the class utilizes a parameter-to-state lookup map as `std::unordered_map<float*, AdamState> _state_by_param_ptr`¹⁴, that provides average constant-time $\mathcal{O}(1)$ lookup complexity. Since weight and bias arrays are allocated once and reside in contiguous memory locations throughout the lifetime of the network, their raw memory addresses (`float*`) serve as unique keys. This pointer-based hashing avoids the overhead of index-based tracking or string-based lookups and enables on-the-fly initialization of the `AdamState` upon the first optimization step.

¹³That accepts predictions, targets and angles of attack (`alphas`) tensors as `const std::shared_ptr<Tensor>&`.

¹⁴Where `AdamState` is a private structure that contains the first moment vector `std::vector<float> m`, the second moment vector `std::vector<float> v`, and an unsigned integer `timestep` to keep track of the parameter update steps.

CNNModel

The concrete class `CNNModel` coordinates the execution, training and state management of the entire neural network. It aggregates the layers of the convolutional branch and the dense branch, manages feature fusion, interfaces with the optimizer and implements utilities for distributed data-parallel training and weight serialization.

The `forward` method propagates input tensors sequentially through the model. To maximize efficiency, the iteration over the layer containers (`conv_blocks` and `dense_blocks`) utilizes `const auto&`, thereby avoiding the performance overhead associated with incrementing reference counters on the underlying shared pointers. Conversely, the `backward` method employs reverse iterators to correctly propagate gradients upstream in strict accordance with the chain rule.

Another interesting detail is that in the `update` method gradient values are clipped to the interval $[-1.0, 1.0]$ to prevent gradient explosion¹⁵. This is executed via a local lambda function (`clip_layer`) containing a nested `clip_tensor` lambda, which directly modifies the elements in-place using standard pointer-based loops.

Whenever layer-specific parameters must be accessed for weight updates, gradient synchronization, or serialization, the model dynamically resolves the abstract base pointer types through runtime downcasting via `std::dynamic_pointer_cast` (e.g., casting from `std::shared_ptr<Layer>` to `std::shared_ptr<Conv2DLayer>`).

Finally, the model implements binary weight export and import methods using `std::ofstream` and `std::ifstream`, opened with the `std::ios::binary` flag. Binary data is written and read directly via type-unsafe low-level operations utilizing `reinterpret_cast<const char*>` and `reinterpret_cast<char*>` to map raw byte buffers, which minimizes I/O latency. During the import phase, after reading the shapes and element counts from the file, weight vectors are populated using `std::move` (from the `<utility>` header) to transfer the ownership of heap-allocated arrays into the target structures, bypassing expensive memory allocation and copying. To illustrate this efficient binary format, Table 4 details the layout of the file header and the metadata corresponding to the initial `Conv2DLayer`.

Table 4: Example layout of the binary weight file header, showing the global count of layers followed by the name and weight/bias tensor metadata of the first layer (`Conv2DLayer`).

Byte Offset	Field Name	Value / Example
0x00-0x07	Total layers in model (N_{layers})	8
0x08-0x0F	Layer name length (L_{name})	39
0x10-0x36	Layer name string	"Conv2DLayer(1,8,5, stride=5, padding=0)"
0x37	Weights presence flag	true (0x01)
0x38-0x3F	Weights tensor rank (R_W)	4
0x40-0x5F	Weights tensor shape	[8, 1, 5, 5]
0x60-0x67	Total weights count (N_W)	200
0x68-0x387	Weights raw data	[-0.0125, 0.0841, ...]
0x388	Biases presence flag	true (0x01)
0x389-0x390	Biases tensor rank (R_B)	1
0x391-0x398	Biases tensor shape	[8]
0x399-0x3A0	Total biases count (N_B)	8
0x3A1-0x3C0	Biases raw data	[0.0, 0.0, ...]

3.2.6. Data Pipelines: NPZDataLoader

The `NPZDataLoader` class manages the retrieval, normalization, and partitioning of datasets stored in the compressed NumPy (`.npz`) format. This format was explicitly selected because it packages multiple named multidimensional arrays within a single ZIP archive, acting as an efficient bridge between Python-based preprocessing scripts (such as `build_dataset.py`) and the C++ training pipeline without the severe parsing overhead

¹⁵This choice has been made after training the model for the first time, where we have noticed very strong fluctuations of the loss value during the training process.

of text-based formats like CSV.

Specifically, the dataset is structured into four core arrays:

- **X_sdf**, representing the 150×150 SDF of the airfoil geometries;
- **X_scalars**, storing the Reynolds number (Re) and the angle of attack (α);
- **Y_cl**, containing the ground-truth lift coefficient (C_L) labels computed from the Navier-Stokes simulations;
- **metadata**, containing the specific NACA airfoil profile labels.

This class provides a complete input pipeline, converting these raw data arrays into normalized, formatted tensor batches for network training.

To import the data without introducing heavy external C++ NumPy parsing libraries, the internal reading method establishes a read-only POSIX pipe to execute a brief Python script, that utilizes Python's native numpy library to load the specified array, converts it to raw single-precision bytes and streams the output directly to the standard output. The C++ program then intercepts this byte stream, reading it sequentially into a standard vector buffer before safely closing the process and verifying its exit status.

Once the arrays are loaded into memory, the constructor applies Z-score standardization across the SDF, scalar parameters and target lift coefficients, to prevent numerical instability during the training phase. Then, the class handles the sequential feeding of this data during training. The batching process partitions the normalized dataset into mini-batches by maintaining an internal cursor that tracks the current read position. For each requested batch, it allocates new tensor objects and utilizes standard library copying algorithms to transfer the exact slice of data from the internal buffers to the newly created tensors.

3.2.7. Main execution of the CNN

The main execution entry point of the neural network workflow is defined in **main.cpp**. It coordinates the loading of datasets, the structural composition of the CNN layers and the training and validation loops over successive epochs.

The program begins by instantiating the training and testing pipelines using **NPZDataLoader** to load the compressed NumPy datasets. It then creates a shared instance of the **AdamOptimizer** wrapped in a **std::shared_ptr** via **std::make_shared** with a designated learning rate of 10^{-5} . This optimizer is passed to initialize the **CNNModel** instance.

The architecture of the neural network is composed using the model's builder methods:

1. the convolutional branch is constructed by adding a **Conv2DLayer** with 1 input channel, 8 output channels, a 5×5 kernel, and a stride of 5, followed by a **LeakyReLULayer** (with an activation slope $\alpha = 0.05$) and a **FlattenLayer** to reduce the spatial feature maps to a one-dimensional representation of size 7200;
2. the feature fusion stage is enabled by registering a **ConcatenateLayer** via **set_concatenate_layer()**, which merges the geometry features with the two physical flow parameters (Reynolds number and angle of attack);
3. the fully-connected regression head is built using three sequential **DenseLayers** of dimensions $7202 \rightarrow 128$, $128 \rightarrow 64$, and $64 \rightarrow 1$, separated by **LeakyReLULayer** activations, resulting in a single scalar prediction representing the lift coefficient (C_L).

The training process iterates for 100 epochs. Within each epoch, it resets the data loaders and processes the dataset in mini-batches. For each batch, it invokes **train_step** on the model instance to run the forward pass, evaluate the PINN loss function, compute parameter gradients via backpropagation and update the weights. After the training phase of each epoch, it runs a validation pass by propagating the validation samples through the network via **forward()** to monitor generalization and check for overfitting by computing the MSE against the validation targets.

3.3. Parallelism

To achieve high-performance capabilities and scale the computational workflow to large-scale grids and datasets, the codebase employs distributed-memory parallelism using the **Message Passing Interface (MPI)** standard.

The parallelization strategy is implemented at two distinct stages: the geometric preprocessing phase (for parallel SDF generation) and the neural network training phase (for distributed data-parallel training).

3.3.1. Parallel SDF Generation

The computation of the SDF, as described in Section 3.1.1, is an embarrassingly parallel task, since the evaluation of the signed distance for each point on the 150×150 grid is independent of all other points.

To do this, upon initializing the MPI environment via `MPI_Init`, the total number of grid points ($N_{\text{total}} = 22,500$) is partitioned by the communicator size (S , retrieved using `MPI_Comm_size`). Because there could be some cases where N_{total} is not perfectly divisible by S , the remainder ($R = N_{\text{total}} \bmod S$) is distributed among the first R processes¹⁶. Each process (with rank ID r , retrieved via `MPI_Comm_rank`) computes its local count and offset as:

$$N_{\text{local}} = \lfloor N_{\text{total}}/S \rfloor + \mathbb{I}(r < R) \quad \text{and} \quad O_{\text{local}} = r \lfloor N_{\text{total}}/S \rfloor + \min(r, R) \quad (44)$$

where $\mathbb{I}$ is the indicator function.

Each process independently computes the SDF values for its assigned local grid points. To reconstruct the complete 2D matrix, the local chunks must be gathered into a global vector of size N_{total} , using `MPI_Allgatherv`, which performs a vector gather-to-all communication. The collective call requires `recvcunts`, which stores the number of elements expected from each rank, and `displs`, which specifies the starting byte displacement of each segment in the global buffer. `MPI_Allgatherv` ensures that all processes receive the full SDF image.

Finally, the root process (rank 0) writes the complete matrix to disk via the `mappingToMatrix` method, while other ranks proceed to the next file or terminate safely via `MPI_Finalize`.

3.3.2. Distributed Data-Parallel Training

The training of the Convolutional Neural Network is parallelized in `CNN/main.cpp` and `CNNModel.cpp` using a data-parallel training strategy to scale to large datasets and decrease training epoch time.

In `CNN/main.cpp`, data parallelism is implemented by distributing the training batch across multiple processes. The global mini-batch size is scaled according to $B_{\text{global}} = B_{\text{local}} \times S$, with a local batch size of $B_{\text{local}} = 64$. To prevent redundant memory duplication, each process uses a custom `slice_batch_tensor` method to extract only its specific segment of the global dataset, beginning at the index offset $r \times B_{\text{local}}$.

To guarantee consistent optimization across the distributed environment, the training pipeline enforces MPI synchronization at three critical stages:

1. initially, the root process generates the network weights and distributes them to all other ranks using `MPI_Bcast` on the raw tensor buffers, ensuring an identical starting state¹⁷ across the entire communicator;
2. during the backpropagation phase, each process computes gradients based on its local batch slice. Before applying the Adam optimization step, these local gradients are aggregated via an in-place `MPI_Allreduce` operation. By utilizing the `MPI_SUM` operator directly on the existing memory buffers, the system efficiently avoids redundant data allocations. The accumulated gradients are then divided by the total number of processes S to compute a global average, which guarantees that every rank applies the exact same mathematical update to its local parameters;
3. finally, at the conclusion of each epoch, the local training losses, validation losses, and processed sample counts are gathered back to the root process via `MPI_Reduce`, allowing it to calculate and report the consolidated global statistics.

3.4. Results

Given the significant model size of 930,513 trainable parameters, performing the training process on standard personal computers was computationally impractical within a reasonable timeframe. To overcome this lim-

¹⁶This partitioning guarantees perfect load balancing, as the workload of any two processes differs by at most a single grid point.

¹⁷With all the weights initialized to the same value.

itation, the training was conducted on the high-performance computing (HPC) resources of the **CINECA Leonardo cluster**. Specifically, we utilized a full single node configured with two Intel Xeon Platinum 8480+ (Sapphire Rapids) processors, providing a total of 112 physical cores, backed by 512 GiB of DDR5 RAM operating at 4800 MHz. This hardware acceleration reduced the total execution time from several hours to approximately five minutes.

```

1 #!/bin/bash
2 #SBATCH --job-name=cnn_training
3 #SBATCH --account=EUHPC_D35_025
4 #SBATCH --partition=dcgp_usr_prod
5 #SBATCH --nodes=1
6 #SBATCH --ntasks-per-node=1
7 #SBATCH --cpus-per-task=112
8 #SBATCH --time=01:00:00
9 #SBATCH --output=cnn_%j.out
10 #SBATCH --error=cnn_%j.err
11
12
13 cd /leonardo/home/userexternal/gdonnine/CNN/
14     Ice_acceleration_model_on_airplane_wing/CNN
15
16 module purge
17 module load python/3.11.7
18 module load gcc/11.3.0
19 module load intel-oneapi-compilers
20 module load intel-oneapi-mpi
21
22 srun ./cnn_mpi_executable

```

Listing 5: SLURM run script for CNN training (`run_cnn.sh`).

The network was trained for 100 epochs on a synthetically generated dataset of 2D SDF, representing five **symmetric NACA airfoil profiles** (NACA 0006, 0008, 0010, 0012 and 0018) across varying angles of attack (α) and Reynolds numbers (Re). Partitioned via an 80/20 split, the dataset yielded 1,713 training samples and 429 validation samples¹⁸.

Initial Unstable Training Run

In our first training attempt, we adopted a standard learning rate of 10^{-3} , without applying input feature normalization or gradient clipping. This configuration resulted in severe numerical instability during the optimization process. As illustrated in Figure 10, the training and validation losses exhibit chaotic, erratic oscillations, fluctuating wildly by over seven orders of magnitude (ranging from 10^{-1} to 10^6 in log scale). The presence of massive loss spikes¹⁹ is a clear indicator of exploding gradients. Without gradient clipping, large backpropagated gradients led to excessive parameter updates, pushing the weights into unstable regions of the loss landscape and preventing the optimizer from settling into a local minimum.

Optimized Successful Training Run

To resolve the numerical instability and ensure smooth convergence, three key modifications were implemented in the training pipeline:

1. the Adam optimizer’s learning rate (η) was reduced from 10^{-3} to 10^{-5} ;
2. standard Z-score normalization was applied during the data-loading phase to scale the inputs (SDF and physical scalars) and targets (lift coefficients) to zero-mean and unit-variance;
3. gradient clipping was enabled in the update phase to clamp all gradient components strictly within the interval $[-1.0, 1.0]$ (as detailed in Section 3.2.5).

These modifications successfully stabilized the training trajectory.

¹⁸All of which have been made publicly available on [Kaggle](#)

¹⁹Such as the prominent peaks around epochs 40, 60, 70 and 85.

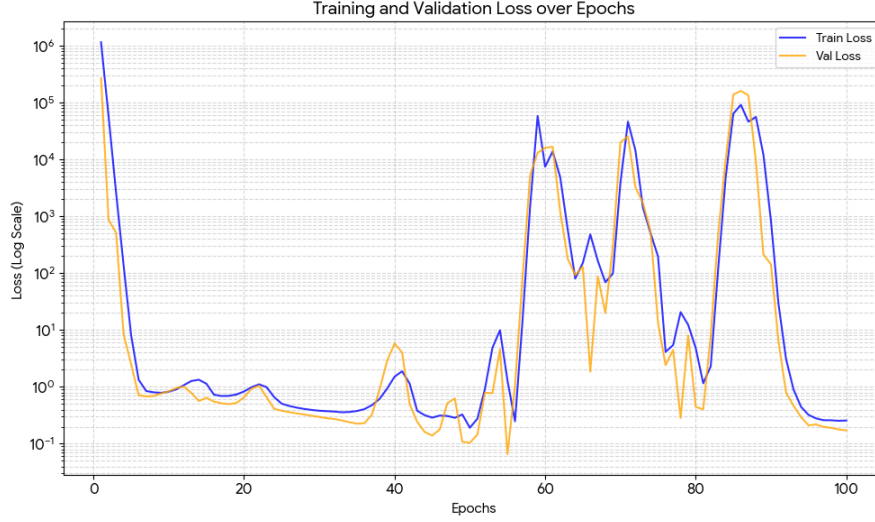


Figure 10: Log-scale evolution of the training and validation losses over 100 epochs during the initial unstable training attempt. The extreme oscillations and sudden spikes up to 10^6 highlight the numerical instability caused by an excessively high learning rate and the absence of gradient clipping or feature normalization.

As shown in Figure 11, the loss curves exhibit highly stable, monotonic convergence. The left plot (Full View) shows the rapid decay of the loss during the initial epochs, indicating that the network quickly learned the dominant physical relationships. The right plot (Zoomed View, Epochs 5-100) highlights the smooth, continuous reduction of both training and validation losses, with no signs of overfitting. The validation loss consistently tracks slightly lower than the training loss; this is a common characteristic when model regularization (such as L2 weight decay) is active during training or when the z-score normalization effectively centers the distribution of test cases.

This optimized training process completed in likely 5 minutes and 11 seconds, achieving a final training loss of 0.0104216 and a validation loss of 0.00784775, demonstrating both the efficiency and accuracy of our implementation.

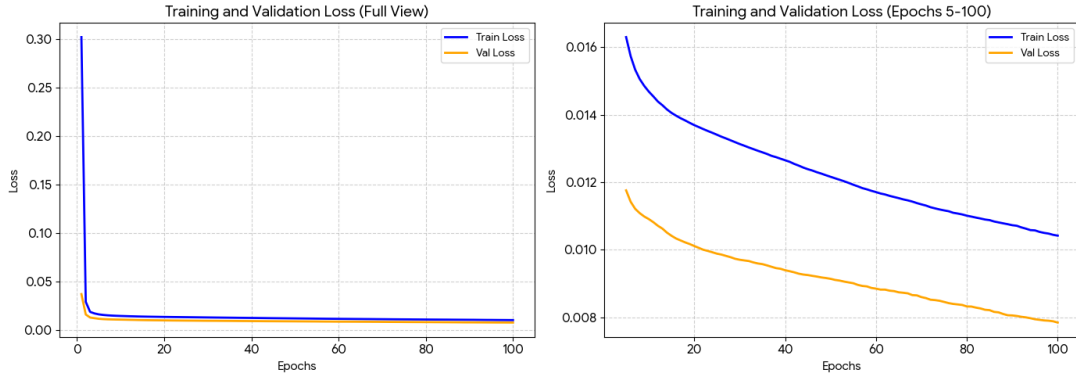


Figure 11: Evolution of training and validation losses for the optimized training run. The full view (left) illustrates the rapid initial convergence, while the zoomed-in view from epoch 5 to 100 (right) highlights the smooth, monotonic decrease of both metrics, confirming the effectiveness of Z-score normalization and gradient clipping.

4. Conclusions

This work successfully demonstrates an integrated approach to aerodynamic analysis, combining high-fidelity numerical simulation with a data-driven surrogate model. The primary achievement was the extension of the

LifeX library to handle turbulent external aerodynamic flows through the integration of the **Spalart-Allmaras RANS model**. This new capability was validated on the **Leonardo HPC cluster**, establishing a robust pipeline for generating reference-quality simulation data. Additionally, a **Convolutional Neural Network** was developed and trained on this data, creating a surrogate model capable of near-instantaneous aerodynamic prediction.

The CFD simulations of the NACA airfoil produced lift coefficients that were physically consistent, though moderately underestimated compared to some experimental data. This behavior is physically consistent with the modeling choices made: the VMS-LES formulation employed in the Navier-Stokes step acts as an implicit filter that dissipates small-scale turbulent structures, suppressing the contribution of fine-grained vortical activity to the overall lift generation. Nonetheless, the results remain well within the range of published numerical and experimental values, confirming the physical validity of the simulation framework by Ladson (1988) and McCroskey (1988).

A further modeling assumption concerns the compressibility of the flow. The simulations were performed at a Mach number significantly below 0.3, a regime in which the relative density variations induced by pressure fluctuations are negligible, making the incompressible Navier-Stokes equations an appropriate model.

Regarding the deep learning implementation, the results demonstrate that a relatively simple Convolutional Neural Network architecture can guarantee robust performance, achieving a Mean Squared Error of 0.0104 on the validation set. The primary advantage of this approach lies in its computational efficiency: while a high-fidelity CFD simulation requires approximately 3 hours on an HPC node, the trained CNN inference time is under one second. Even when accounting for the 5 minutes and 11 seconds required for training, this deep learning framework achieves a remarkable $35.11\times$ speedup compared to conventional methods. This outcome strongly aligns with the premises established by Cuozzo et al. (2026), extending their insights to regular profiles, whereas their approach specifically accounted for deformed airfoils due to ice accretion.

This contrast between computational cost and prediction speed places our work within the context of modern aerodynamic design. From a broader perspective, the high-fidelity approach developed here occupies the opposite end of the cost spectrum from purely surrogate-based methods. For instance, the framework proposed by Cuozzo et al. (2026) achieves a $10^3 - 10^4\times$ reduction in wall-clock time at the cost of a $\sim 14\%$ underestimation of the iced lift curve slope. The present work, instead, prioritizes physical fidelity to provide the reference-quality data that such surrogate models ultimately depend on for training and validation. The two approaches are therefore complementary rather than competing, and the framework developed here constitutes a natural candidate as a high-fidelity data source for future surrogate-based pipelines targeting iced airfoil configurations.

Nevertheless, certain limitations inherent to each method must be acknowledged. These include the computational demand of the RANS simulations and, for the data-driven model, the absence of an *a priori* error estimate, the strict dependency on a training dataset that accurately resembles the target inference domain²⁰, and the general lack of interpretability characteristic of black-box models.

Looking forward, the framework developed here opens up promising avenues for engineering design and analysis. The speed of the trained CNN makes it ideal for real-time applications and rapid design iterations on consumer-grade hardware, completely bypassing the need for constant HPC access. This capability is particularly relevant for the agile aerodynamic design and field-testing of components for Unmanned Aerial Vehicles and quadcopters, where on-the-fly performance evaluation can accelerate development cycles. The simulation pipeline itself is now poised to serve as a data engine for training future surrogate models on more complex geometries, such as the iced airfoils that motivated this study.

References

P. C. Africa, I. Fumagalli, M. Bucelli, A. Zingaro, M. Fedele, L. Dedè, and A. Quarteroni. lifex-cfd: An open-source computational fluid dynamics solver for cardiovascular applications. *Computer Physics Communications*, 296:109039, 2024.

Ciro Cuozzo, Vincenzo Cusati, Agostino De Marco, Salvatore Corcione, James H. Page, and Alessandro Zanon.

²⁰As noted, since the training data from the CFD was slightly biased, we expect this bias persists in the surrogate model.

Ai-driven surrogate modeling for iced tailplane aerodynamic prediction. *Aerospace Science and Technology*, 168, 2026.

Charles L. Ladson. Effects of independent variation of Mach and Reynolds numbers on the low-speed aerodynamic characteristics of the NACA 0012 airfoil section. Technical Report NASA TM-4074, NASA Langley Research Center, 1988.

W. J. McCroskey. A critical assessment of wind tunnel results for the NACA 0012 airfoil. Technical Report NASA TM-100019, NASA Ames Research Center, 1988.

Philippe R. Spalart and Steven R. Allmaras. A one-equation turbulence model for aerodynamic flows. *AIAA Paper*, 92-0439, 1992.